COMPLETE STUDY GUIDE

Spring Boot Masterclass

Annotations · Layers · Flow · Database

Ek dum seedha aur simple language mein — sab kuch step-by-step diagram aur flowchart ke saath samjhao

Annotations | MVC Flow | Service Layer | Repository | Security | Database | XML Config | application.yml | Static Files | Flowcharts

Table of Contents

Is PDF mein kya kya hai — ek nazar mein

1 Spring Boot kya hota hai? — Big Picture Overview 3

2 Important Annotations — Part 1 (Core, Startup, Controller) 4

3 Important Annotations — Part 2 (JPA, Service, Request) 5

4 MVC Architecture aur Request Flow Diagram 6

5 Controller Layer — Kaam aur Code 7

6 Service Layer — Business Logic ka Ghar 8

7 Repository Layer — Database se Baat 9

8 Model / Entity — Data ka Blueprint 10

9 Security Config — Login aur Authorization 11

10 application.properties / application.yml 12

11 XML Files — pom.xml aur Configuration 13

12 Database Connection — JPA/Hibernate ka Flow 14

13 Static Files — HTML, CSS, JS ka Flow 15

14 Complete End-to-End Flow Diagram 16

Tip: Pehle Section 1 aur 4 padho — big picture samajh aao toh baaki sab automatically connect ho jayega bhai!

1 Spring Boot Kya Hota Hai?

Big Picture — Seedhi baat, no bakwaas

Bhai, **Spring Boot** basically ek Java framework hai jo tumhara kaam aasaan kar deta hai. Normally Spring mein bahut saari settings manually karni padti thi. Spring Boot ne yeh sab **auto-configure** kar diya.

Seedha example: Spring = sab kuch khud se setup karna. Spring Boot = sab pehle se ready hai, bas apna kaam shuru karo!

SPRING BOOT — 3 MAIN PILLARS

Auto-Configuration
Khud hi setup karta hai

→

Embedded Server
Tomcat andar hi hai

→

Starter POMs
Dependencies ready-made

Project ka Structure kaise dikhta hai?

```
my-spring-app/
src/main/java/com/example/
  MyApplication.java        // @SpringBootApplication (Entry Point)
  controller/
    UserController.java     // @RestController
  service/
    UserService.java        // @Service
  repository/
    UserRepository.java     // @Repository
  model/
    User.java               // @Entity
  config/
    SecurityConfig.java     // @Configuration
src/main/resources/
  application.properties    // DB connection, port, etc.
  static/
    index.html   style.css   app.js
pom.xml                      // Dependencies yahan hain
```

Browser / Client (User)

HTTP Request bhejta hai — GET, POST, PUT, DELETE

↕ HTTP / REST API

Controller Layer

@RestController @GetMapping @PostMapping

Request receive karta hai, Service ko deta hai

↕ Method Call

Service Layer

`@Service @Transactional`

Business logic yahan hoti hai

↕ Method Call

Repository Layer

`@Repository JpaRepository`

Database se data nikalta / dalta hai

↕ JDBC / JPA / Hibernate

Database (MySQL / PostgreSQL)

`Tables · Rows · Columns`

Actual data yahan store hota hai

2 Important Annotations — Part 1

Core, Startup aur Controller Annotations

Annotation kya hota hai? — Bhai, annotation ek special instruction hai jo class ya method ke upar `@` sign ke saath likhte ho. Spring Boot yeh padh ke samajhta hai ki "isko kaisa handle karna hai".

Core / Startup Annotations

@SpringBootApplication MOST IMPORTANT

Yeh teen annotations ka combo hai: @Configuration + @EnableAutoConfiguration + @ComponentScan. Main class ke upar lagta hai. Yahi Spring Boot ka "Start Engine" button hai. Iske bina app start nahi hogi.

@Component

Kisi bhi Java class ko Spring ka Bean banana ho toh @Component lagao. Spring is class ka object khud banata aur manage karta hai. @Service, @Repository, @Controller — yeh sab @Component ke hi special versions hain.

@Autowired

Dependency Injection ke liye. Matlab — ek class ko doosri class ki zaroorat hai, toh manually new karne ki jagah @Autowired lagao aur Spring khud inject kar dega. Jaise — microwave mein plug lagao, bijli automatically aayegi!

@Bean

@Configuration class ke andar methods pe lagta hai. Jab tum third-party library ki class ko Spring bean banana chahte ho, tab @Bean use karte ho.

@Configuration

Is class mein application ki configuration hogi — Database settings, Security settings, Custom beans. Yeh batata hai ki "yeh class setting-wali class hai".

Controller Annotations

@Controller

MVC pattern mein HTML page return karta hai. Traditional web apps mein use hota tha.

@RestController Modern

@Controller + @ResponseBody. JSON/XML data return karta hai. REST API banane ke liye yahi use karo.

@GetMapping

HTTP GET request handle karta hai. Data fetch karne ke liye (list dikhana, ek user dekhna).

@PostMapping

HTTP POST request. Naya data create karne ke liye (new user banana, form submit).

`@PutMapping`

HTTP PUT request — pura record update karna ho toh yeh use karo.

`@DeleteMapping`

HTTP DELETE request — koi record delete karna ho.

`@PatchMapping`

Sirf kuch fields update karne ke liye. PUT poora object replace karta hai, PATCH sirf kuch cheez.

`@RequestMapping`

Base URL define karta hai — /api/users. Yeh sab methods ke liye common prefix ban jaata hai.

3 Important Annotations — Part 2

Request Handling, JPA/Entity aur Service Annotations

Request se Data Lene ke Annotations

`@PathVariable`

URL ke andar se value nikalta hai. /users/5 → @PathVariable Long id → id = 5 aa jaayegi.

`@RequestParam`

URL ke query string se value. /search?name=rahul → @RequestParam String name → "rahul" mil jaayega.

`@RequestBody`

POST/PUT ke body mein jo JSON aata hai usse Java object mein convert karta hai.

`@ResponseBody`

Java object ko JSON mein convert karke browser ko bhejta hai. @RestController mein yeh automatically hota hai.

`@RequestHeader`

HTTP header se value nikalta hai. Authorization token, Content-Type, etc.

`@CrossOrigin`

CORS enable karta hai — different domain se (jaise React frontend) API call allow karne ke liye.

Database / JPA Annotations

`@Entity` JPA

Yeh Java class ek Database Table hai — yeh batata hai. Bina iske JPA class ko table nahi manega.

`@Table(name = "users")`

Table ka naam specify karna ho toh. Agar nahi likha toh class ka naam hi table naam banega.

`@Id`

Primary Key field ko mark karta hai. Har Entity mein ek @Id zaroori hai.

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

ID auto-increment hogi — manually set nahi karna. Database khud badhata rahega (1, 2, 3...).

`@Column(name="email", nullable=false, unique=true)`

Column properties — naam, null allowed hai ya nahi, unique chahiye ya nahi.

Service / Transaction Annotations

`@Service`

`@Transactional`

Business logic wali class. @Component ka hi naam hai — Spring ko pata chale yeh service layer hai.

Database operations ek transaction mein wrap karta hai. Koi error aaye toh sab rollback. Bank transfer jaisa — ya poora ho ya kuch nahi.

`@Repository`

Database access layer. Database exceptions ko Spring exceptions mein translate karta hai.

`@Value("${prop.key}")`

application.properties se value inject karta hai. @Value("${server.port}") → 8080 aa jaayega.

4 MVC Architecture + Request Flow

Browser se Database tak — poora safar ek diagram mein

Bhai, MVC matlab **Model-View-Controller**. Kaam alag alag layers mein banta hai. Jaise restaurant mein waiter alag, chef alag, ingredients store alag — sab ka kaam alag lekin sab connected hain.

COMPLETE HTTP REQUEST FLOW — STEP BY STEP

Browser / Postman
GET /api/users/1 bheja

↓ HTTP Request

DispatcherServlet
Spring ka Front Controller

↓ Route dhundho

HandlerMapping
Kaunsa Controller handle karega?

↓ @GetMapping match hua

@RestController
UserController.getUser(1) call hua

↓ Service ko kaam do

@Service
UserService.findById(1)

↓ Data chahiye toh Repository se

@Repository
UserRepository.findById(1)

↓ SQL Query execute

Database
SELECT * FROM users WHERE id=1

STEP-BY-STEP SUMMARY

Step 1 — Request Aai
Browser ne GET request bheji

Step 2 — DispatcherServlet
Sab requests ka darwaza

Step 3 — Controller
@GetMapping se match hota hai

Step 4 — Service
Business logic execute hoti hai

Step 5 — Repository
Database se data fetch/save

Step 6 — Database
SQL query chalti hai

Step 7 — Response
JSON bana ke browser ko bhej diya

↑ Data wapas aaya

JSON Response

{"id":1,"name":"Rahul"} browser ko mila

5 Controller Layer

Request receive karo, response do — bas itna kaam hai

Controller layer traffic police jaisi hai bhai. Har request aati hai iske paas — yeh decide karta hai kahan bhejni hai. Iska kaam sirf routing hai — logic nahi, database nahi.

```
@RestController                                // JSON response dega
@RequestMapping("/api/users")              // Base URL
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping                                // GET /api/users
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    @GetMapping("/{id}")                    // GET /api/users/5
    public User getUserById(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PostMapping                               // POST /api/users
    public User createUser(@RequestBody User user) {
        return userService.save(user);
    }

    @DeleteMapping("/{id}")
    public void deleteUser(@PathVariable Long id) {
        userService.deleteById(id);
    }
}
```

KONSA HTTP METHOD KAB USE KARO?

@GetMapping Data fetch karo GET /users	**@PostMapping** Naya data banao POST /users	**@PutMapping** Pura update karo PUT /users/1
@PatchMapping Thoda update PATCH /users/1	**@DeleteMapping** Data hatao DELETE /users/1	**@RequestMapping** Base URL set /api/v1

Controller mein koi heavy logic mat likho — sirf request receive karo aur Service ko call karo!

6 Service Layer

Business Logic ka ghar — yahan asli kaam hota hai

Service layer woh jagah hai jahaan asli kaam hota hai. Controller ne request receive ki aur yahan bhej di. Yahan validation, calculation, email bhejna — yeh sab Service ka kaam hai.

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Transactional(readOnly = true)
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    @Transactional
    public User createUser(User user) {
        // Business Logic — Email check karo
        if (userRepository.existsByEmail(user.getEmail())) {
            throw new RuntimeException("Email already exists bhai!");
        }
        // Password encrypt karo
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        return userRepository.save(user);
    }
}
```

@TRANSACTIONAL KAISE KAAM KARTA HAI — BANK TRANSFER EXAMPLE

Transaction Start → A ka balance -1000 → B ka balance +1000 → COMMIT ✓

Agar beech mein error aaya toh ↓

ROLLBACK — Sab wapas A ka bhi, B ka bhi

7 Repository Layer

Database se seedha baat karna — JpaRepository magic

Repository layer ka kaam hai database se data lena aur dena. Spring Data JPA ke saath sirf interface banana hai — baaki Spring khud kar deta hai!

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // In-built methods milte hain FREE mein:
    // findAll()        SELECT * FROM users
    // findById(id)     SELECT * FROM users WHERE id = ?
    // save(user)       INSERT / UPDATE
    // deleteById(id)   DELETE FROM users WHERE id = ?
    // count()          SELECT COUNT(*) FROM users

    // Custom methods — naam se hi query ban jaati hai!
    User findByEmail(String email);
    // SELECT * FROM users WHERE email = ?

    List<User> findByNameContaining(String name);
    // SELECT * FROM users WHERE name LIKE '%name%'

    boolean existsByEmail(String email);

    // Agar khud SQL likhna ho:
    @Query("SELECT u FROM User u WHERE u.age > :age")
    List<User> findUsersOlderThan(@Param("age") int age);
}
```

JPAREPOSITORY — KYA KYA MILTA HAI FREE MEIN?

findAll() Sab records lao	**findById(id)** ID se ek record	**save(entity)** Insert ya Update
saveAll(list) Batch insert	**deleteById(id)** ID se delete	**count()** Kitne records hain
existsById(id) Hai ya nahi?	**deleteAll()** Sab delete	**findAll(Pageable)** Pagination

Magic Naming: findByEmailAndName(email, name) — sirf naam likhdo, JPA khud SQL query bana deta hai!

8 Model / Entity

Data ka blueprint — Java class = Database Table

Model ya Entity ek Java class hai jo database ki table ko represent karti hai. @Entity lagao toh Hibernate khud table banana jaanta hai!

```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Order> orders;   // Ek User ke many Orders
}
```

TABLE RELATIONSHIPS

User
id, name, email

→ @OneToMany
1 User = Many Orders
← @ManyToOne

Order
id, amount, userId

↔ @ManyToMany
Student <-> Course

Course
id, title, students

Lombok use karo — @Data, @Getter, @Setter annotations se getters/setters automatically generate ho jaate hain!

9 Security Config

Login, Logout, Authorization — darwaza kaun khol sakta hai?

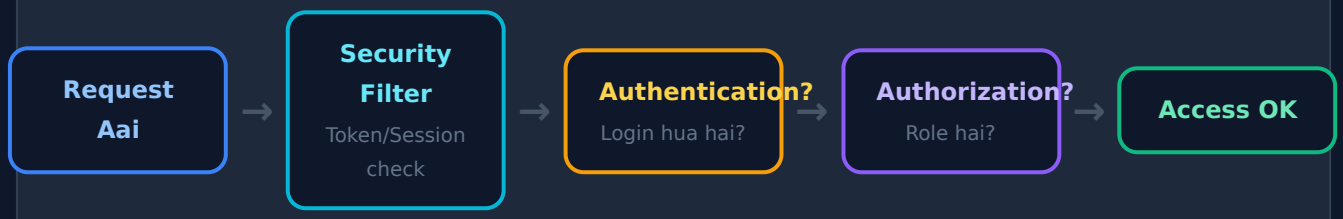
Spring Security application mein security ki layer add karta hai. Kaunsa URL sab dekh sakte hain, kaunse ke liye login chahiye — yeh sab yahan define hota hai.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()  // Sab dekh sakte
                .requestMatchers("/api/admin/**").hasRole("ADMIN") // Sirf ADMIN
                .anyRequest().authenticated()
            )
            .formLogin(login -> login.loginPage("/login").permitAll())
            .logout(logout -> logout.permitAll());
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

SECURITY FILTER CHAIN — REQUEST CHECK HOTI KAISE HAI?



Authentication fail → 401 Unauthorized | Authorization fail → 403 Forbidden

Authentication — Kaun hai?

Login karna. Username/Password check karna. JWT token verify karna. "Pehchano khud ko".

Authorization — Kya kar sakta hai?

Login ke baad bhi check — admin hai, user hai? Role-based access control.

10 application.properties / application.yml

App ki settings — database, port, server sab yahan

Yeh file Spring Boot application ki settings ki file hai. Port number, database URL, passwords — sab yahan likhte hain. Code mein hard-code nahi karte.

application.properties

```
# Server
server.port=8080
spring.application.name=my-app

# Database
spring.datasource.url=jdbc:mysql://
  localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret123

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Logging
logging.level.root=INFO
```

application.yml (same, different format)

```yaml
server:
  port: 8080

spring:
  application:
    name: my-app
  datasource:
    url: jdbc:mysql://localhost/mydb
    username: root
    password: secret123
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

ddl-auto Options — Kya Karta Hai?

create

Har baar app start hone pe tables dobara bana deta hai. Data delete ho jaata hai! Sirf dev mein.

update (Dev ke liye best)

Naye columns add kar deta hai, purane rakhta hai. Data safe rehta hai.

validate

Sirf check karta hai tables sahi hain ya nahi. Kuch change nahi karta.

none (Production)

Kuch nahi karta — tables manually banana padega. Production mein yahi use karo.

Production mein ddl-auto=create kabhi mat lagana — sara data delete ho jaayega!

11 XML Files — pom.xml

Dependencies ki list — project ka ingredients

pom.xml (Project Object Model) tumhare project ki ingredients list hai. Jaise recipe mein likha hota hai kya kya chahiye — pom.xml mein kaunsi libraries chahiye yeh likha hota hai.

```xml
<!-- pom.xml -->
<project>
    <groupId>com.example</groupId>
    <artifactId>my-app</artifactId>
    <version>1.0.0</version>

    <parent>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.0</version>
    </parent>

    <dependencies>
        <!-- Web / REST API -->
        <dependency>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- JPA / Database -->
        <dependency>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- MySQL Driver -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
        </dependency>

        <!-- Security -->
        <dependency>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
    </dependencies>
</project>
```

COMMON SPRING BOOT STARTERS

spring-boot-starter-web
REST API + Tomcat server

spring-boot-starter-data-jpa
JPA + Hibernate

spring-boot-starter-security
Login/auth system

spring-boot-starter-validation
@Valid, @NotNull input validation

spring-boot-starter-test
JUnit + Mockito testing

spring-boot-starter-actuator
App health check aur monitoring

12 Database Connection — JPA/Hibernate ka Flow

Java se Database tak — poora connection ka safar

Bhai, database connect karna Spring Boot mein bahut aasaan hai. Bas 3 cheezein chahiye: dependency add karo, properties mein URL likho, Entity aur Repository banao — ho gaya!

DATABASE CONNECTION — STEP BY STEP

1. pom.xml
mysql-connector + data-jpa add karo

→

2. Properties
DB URL, username, password

→

3. @Entity class
Table ka blueprint

→

4. Repository
JpaRepository extend

Bas! Spring Boot khud connection pool banata hai, Hibernate SQL generate karta hai

Connection ke andar kya hota hai?

Java Code
repository.save(user)

→

JPA Interface
Abstraction layer

→

Hibernate ORM
Java ↔ SQL translate

→

JDBC Driver
Network connection

→

MySQL DB
INSERT done!

ORM kya hai?

Java ka User class = Database ki users table. Hibernate yeh mapping karta hai. Tum Java objects ke saath kaam karo — SQL Hibernate likhta hai.

Connection Pool (HikariCP)

Har request pe nayi DB connection banana slow hota hai. HikariCP connections pool mein ready rakhta hai. Spring Boot mein automatically configure hota hai.

13 Static Files — HTML, CSS, JS

Frontend files Spring Boot mein kahan rakho aur kaise serve hoti hain

Static files woh files hain jo server process nahi karta — seedhe browser ko bhejta hai. HTML, CSS, JS — yeh sab static files hain. src/main/resources/static/ folder mein jaati hain.

```
src/main/resources/
  application.properties
  static/                 Yahan rakho static files
    index.html          http://localhost:8080/index.html
    about.html
    css/
      style.css           http://localhost:8080/css/style.css
    js/
      app.js              http://localhost:8080/js/app.js
    images/
      logo.png
  templates/              Thymeleaf dynamic templates
    home.html
```

STATIC VS DYNAMIC — FARAK KYA HAI?

Static File (HTML/CSS/JS)	Dynamic (Thymeleaf)
Browser ne /index.html maanga	Browser ne /home maanga
↓	↓
Spring static folder mein dhundha	Controller → Service → DB
↓	↓
File seedhe bhej di — as-is Koi processing nahi	Template + Data = HTML Server pe process

JS se Spring Boot API Call karna

```javascript
// app.js — Frontend se Backend API call
fetch('/api/users')
    .then(response => response.json())
    .then(users => {
        users.forEach(user => {
            document.getElementById('list')
                .innerHTML += '<li>' + user.name + '</li>';
        });
    });
```

```
// POST — Form submit karna
fetch('/api/users', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name: 'Rahul', email: 'r@r.com' })
});
```

14 Complete End-to-End Flow

Register button dabaya — ab kya hoga? Step by step

USER NE "REGISTER" BUTTON DABAYA — POORA FLOW

1. **Browser — POST /api/register**
   Form ka data JSON mein pack hota hai: {"name":"Rahul","email":"r@r.com","pass":"123"}

2. **Security Filter Chain**
   /api/register → permitAll() → aage jane do, login ki zaroorat nahi

3. **DispatcherServlet @RestController @PostMapping**
   @RequestBody ne JSON ko User object mein badla → userService.register(user) call

4. **@Service @Transactional — Business Logic**
   Email already hai? Error. Nahi → Password BCrypt se hash karo → Repository ko save karo

5. **@Repository JpaRepository.save(user)**
   Hibernate ne SQL banaya: INSERT INTO users (name, email, password) VALUES (?, ?, ?)

6. **MySQL Database — INSERT execute hua**
   JDBC Driver ne query bheji, database ne execute ki, auto-generated ID wapas aaya (id=42)

7. **Response — Browser ko mila**
   HTTP 201 Created + JSON: {"id":42,"name":"Rahul"} → Browser mein success dikha!

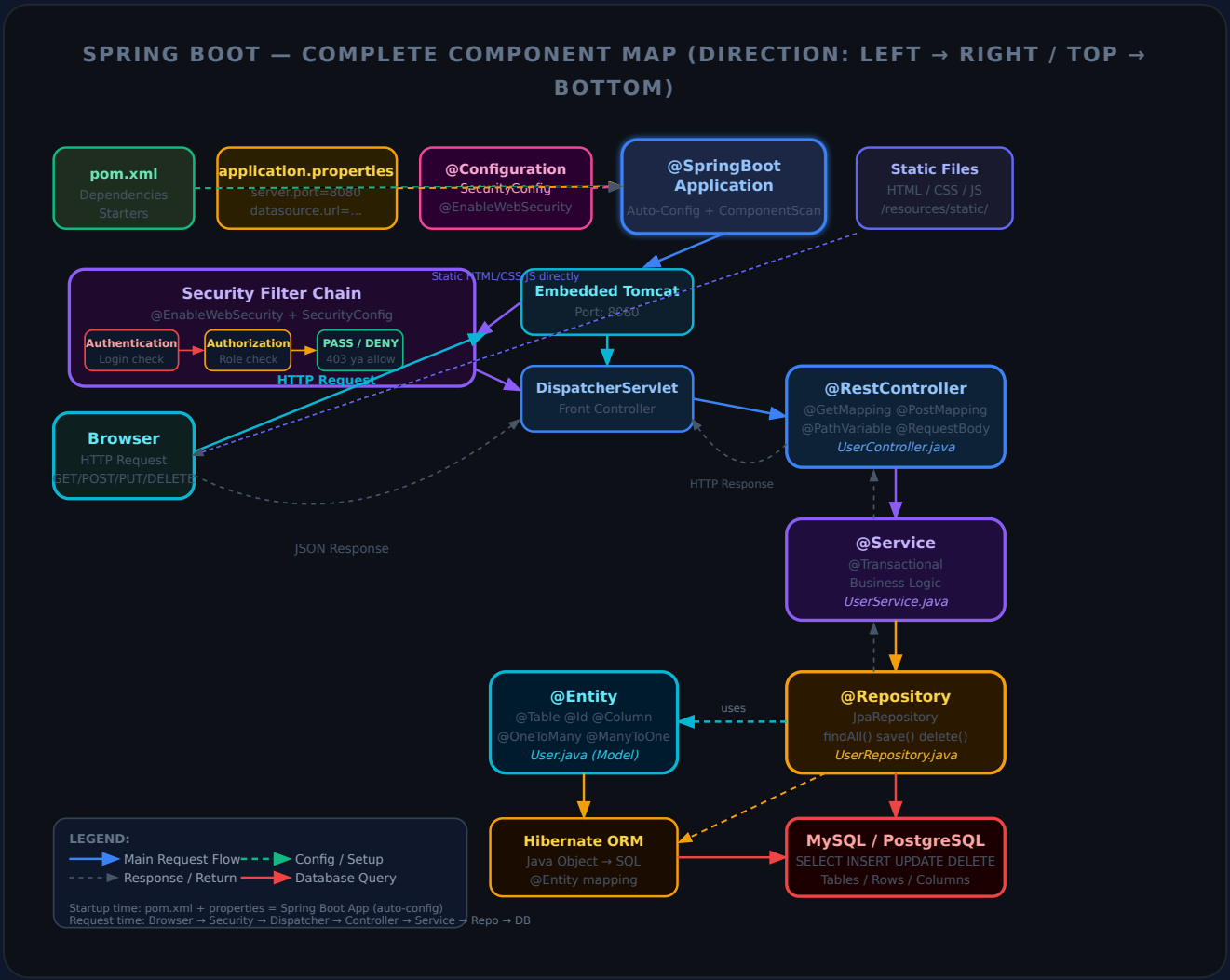
Summary: Browser → Security → DispatcherServlet → Controller → Service → Repository → Database → wapas response. Yeh Spring Boot ka dil hai!

Spring Boot Mast Hai Bhai!

Annotations yaad karo → MVC flow samjho → Layers alag rakho → Sab set hai!

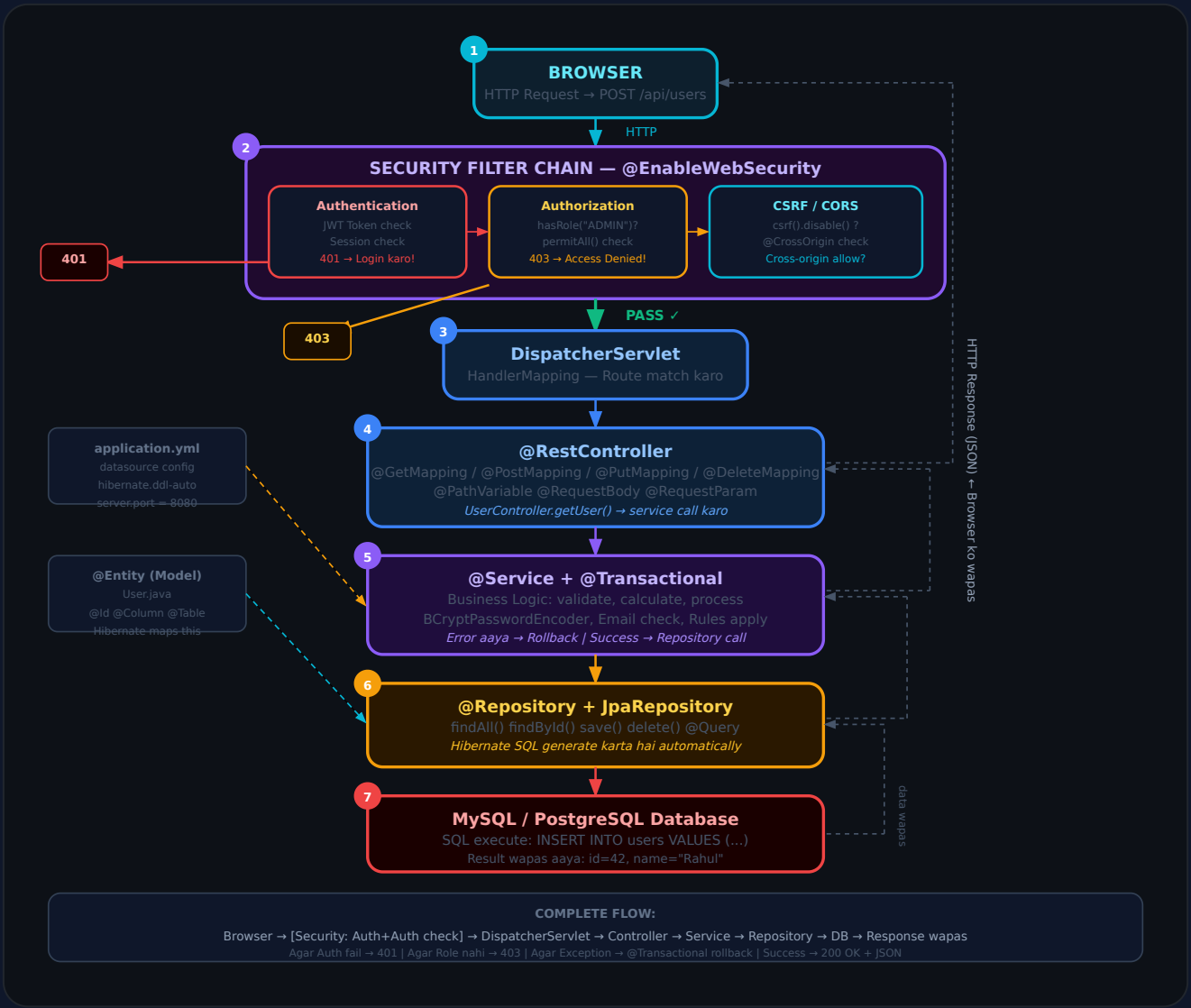
15 Full Architecture — Sab Ek Saath

Har component kahan fit hota hai — complete map with annotations



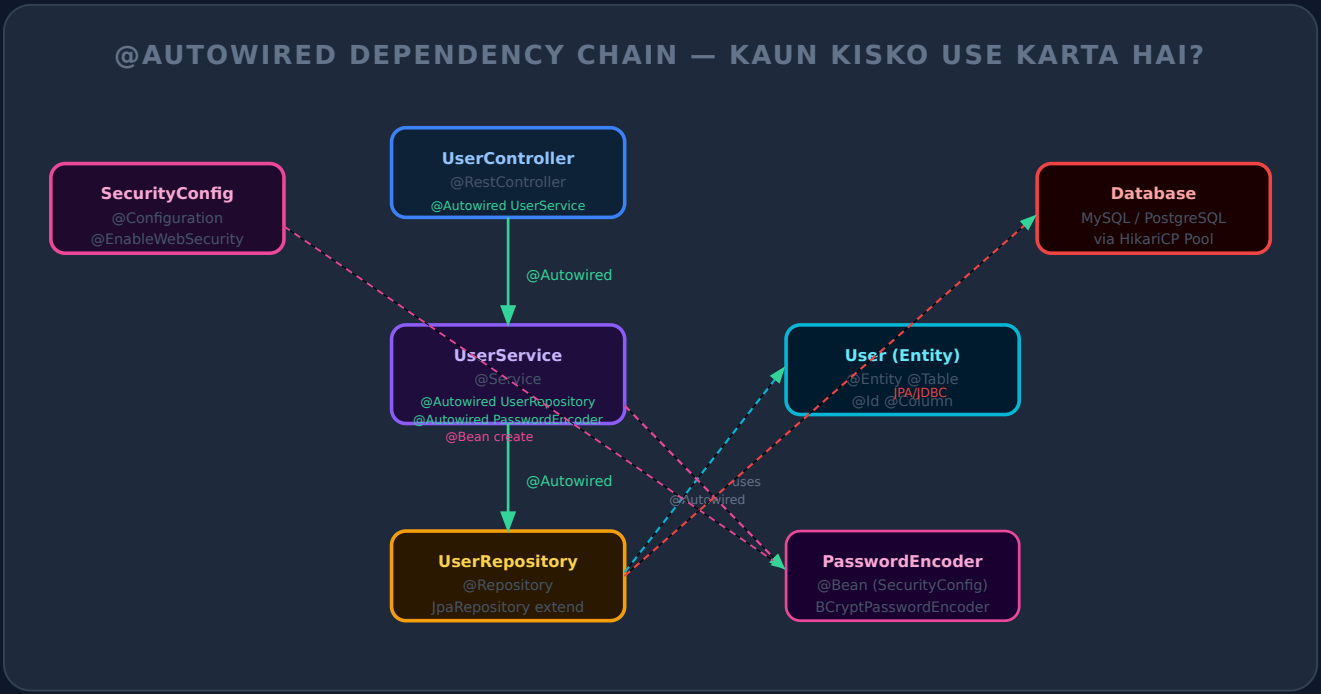
16 Security + Full Request — Detail Flowchart

Har decision point, har check — Security ke saath complete flow



17 Component Connection Summary

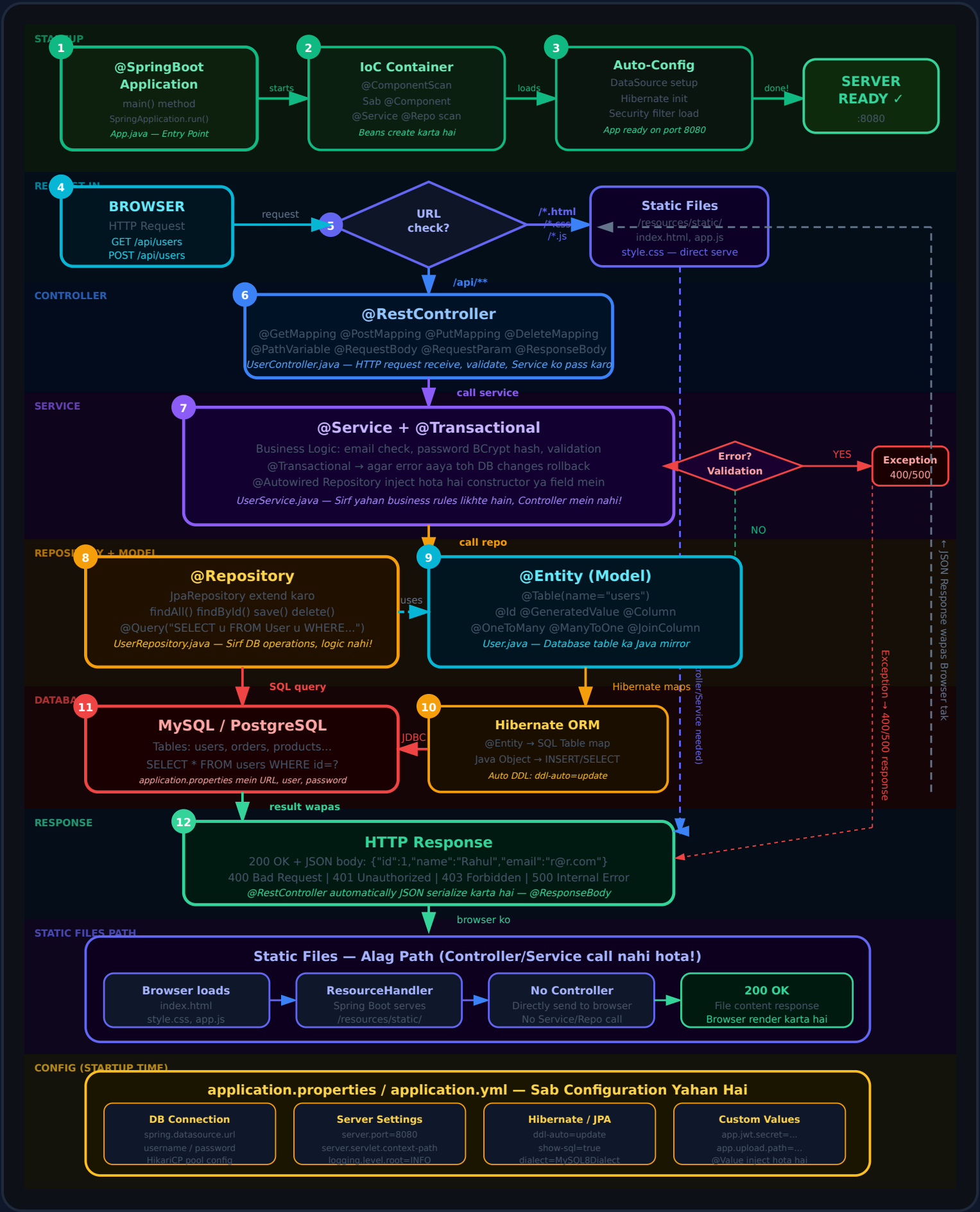
Konsa component kisse inject hota hai — @Autowired connections



Quick Reference — Kaun Kahan Hota Hai?

File / Class	Annotation	Kaam	Inject Karta Hai
`UserController.java`	`@RestController`	HTTP request receive karo	`UserService`
`UserService.java`	`@Service`	Business logic execute karo	`UserRepository, PasswordEncoder`
`UserRepository.java`	`@Repository`	Database CRUD operations	`JPA / Hibernate (auto)`
`User.java`	`@Entity`	DB table ka blueprint	Kuch inject nahi karta
`SecurityConfig.java`	`@Configuration`	Security rules define karo	`PasswordEncoder @Bean`
`application.properties`	`@Value inject`	DB URL, port, settings	Spring auto-configures
`MyApplication.java`	`@SpringBootApplication`	App start karo	Sab kuch scan + configure

Final Rule: pom.xml mein library → properties mein settings → @Entity se table → @Repository se DB access → @Service mein logic → @Controller se HTTP handle → Security ne gate pe rokke rakha. Yeh Spring Boot hai bhai!

18 Master Flowchart — Sab Components Ek Saath

Application → Controller → Service → Repository → Model → Database + Static Files — kab kya call hota hai